

Note-Binding Verifier Implementation Status

Perry Kundert

2026-06-10

Contents

1	Overview	1
2	Completed	2
2.1	idHash Layout Reconciliation	2
2.2	Circuit Design (<code>circuits/note_binding.circom</code>)	2
2.3	BN254 G-Generator Powers Table	3
2.4	Solidity Contracts	3
2.5	Tests	3
2.6	A1-Layout Circuit (<code>circuits/note_binding_a1.circom</code>)	3
2.7	Supporting Scripts	4
2.8	Witness Generation — ROOT CAUSE FOUND AND FIXED	4
2.8.1	Diagnosis trail	4
2.8.2	Fix	5
2.9	rapidsnark Integration	5
2.10	Groth16 Trusted Setup — COMPLETE; two more toolchain discoveries	5
2.10.1	Discovery 1 — snarkjs sizes the Groth16 domain on TOTAL constraints	6
2.10.2	Discovery 2 — snarkjs 0.7.5 exports the VK already EIP-197-ordered	6
2.10.3	Final results	6
3	Remaining Issues	6
3.1	flake.nix Dependencies	6
4	File Inventory	7
5	Build Commands	7

1 Overview

The `note<->eEnc` re-encryption tie (`INoteBindingVerifier`) closes two gaps the flavor-agnostic spend proof leaves open:

1. **addressed-binding** — only the recipient Identity M_{rec} can spend an A1/A2 note (without it, any registered account holding the opening can redeem with a self-addressed eEnc).
2. **A2 collusion** — an un-nameable note is un-spendable (without it, a colluding pair can substitute a different, nameable eEnc).

Commit 1e410a6 plumbed the interface and a `StubNoteBindingVerifier` into the A1/A2 spend paths. The real ZK circuit and verifier are being implemented under the branch `feature/decryptability`.

2 Completed

2.1 idHash Layout Reconciliation

Two competing A2 idHash layouts existed in the repo:

- `mint_batch_a2.circom` and `wallet/notes.py` used `Poseidon8(eNote, eIss)`
- `wallet/unilateral_a2.py` used `Poseidon4(eIss)`

Fixed `unilateral_a2.py::a2_id_hash` to delegate to `notes.id_hash_a2(eNote, eIss)` and updated all callers. Also restructured `unilateral_a1.py` to use `id_hash_a1` (8-input Poseidon) and generate `eNote` internally, matching the pattern now used by the A2 path.

Files changed:

- `alberta_buck/wallet/unilateral_a2.py` — `a2_id_hash`, `MintedA2`, `mint_unilateral_a2`
- `alberta_buck/wallet/unilateral_a1.py` — `MintedA1`, `mint_unilateral_a1`
- `alberta_buck/test/test_unilateral_a2.py` — test callers
- `alberta_buck/test/test_unilateral_a1.py` — test callers

All 29 Python wallet tests pass.

2.2 Circuit Design (`circuits/note_binding.circom`)

The initial design used `EllipticCurvePippengerMult` for variable-base scalar multiplication (`m_rec * R0`), producing ~6.3M non-linear constraints and a 1.8 GB R1CS file — too large for practical use.

Redesigned to exploit the ElGamal structure of `eIssCommitted`:

- $R0 = r * G$ and $C0 = M_I + r * M_{rec}$ where M_I is the decrypted issuer identity.
- Instead of computing $m_{rec} * R0$ (variable-base), the prover witnesses M_I and r , and the circuit verifies the ElGamal structure using only fixed-base scalar multiplications ($r * G$ and $(r * m_{rec}) * G$).

Result: ~2.4M non-linear constraints, 716 MB R1CS, 25 public inputs, 58 private inputs. The circuit compiles successfully with circom 2.2.3.

Circuit constraints:

Constraint	Description	EC ops
(1)	$\text{nullifier} = \text{Poseidon3}(\text{rho}, \text{idHash}, 4242)$	—
(2)	$\text{idHash} = \text{Poseidon8}(\text{eNote}, \text{eIss0})$	—
(3a)	$\text{eEnc.R} = R0 + s * G$	$s * G$ (fixed-base)
(3b)	$\text{eEnc.C} = C0 + (s * m_{rec}) * G$	$(s * m_{rec}) * G$ (fixed-base)
(4a)	$R0 = r * G$	$r * G$ (fixed-base)
(4b)	$C0 = M_I + (r * m_{rec}) * G$	$(r * m_{rec}) * G$ (fixed-base)
(4c)	$P_I = M_I + b * H$	$b * H$ (fixed-base)

Scalar products $sm = s * m_{rec}$ and $rm = r * m_{rec}$ are witnessed in 4-limb form; the circuit constrains them against the native-field products.

2.3 BN254 G-Generator Powers Table

The `circom-lib` optimized scalar multiplication (`EllipticCurveScalarGeneratorMultiplicationOptimised`) has precomputed stride-8 power tables for `secp256k1`, `brainpoolP256r1`, and `P256`, but **not** for `BN254`. Without a matching table, the template silently produces garbage (all-zero powers $\rightarrow$ points at infinity).

Generated `circuits/ec/powers/bn254_g_pows.circom` (8214 lines, 2.7 MB) containing the full $32 \times 256 \times 2 \times 4$ precomputed powers table for $G=(1,2)$ on `BN254`, and created `circuits/ec/bn254_g_scalarmul.circom` (`ScalarMulG()`) mirroring the existing `ScalarMulH()` template.

Makefile target: `snark-g-pows` regenerates the powers file from the Python wallet’s EC primitives.

2.4 Solidity Contracts

- `src/NoteBindingVerifierAdapter.sol` — wraps the generated Groth16 verifier behind `INoteBindingVerifier`. Unpacks proof bytes into a Groth16 triple (a, b, c) , decomposes 6 point coordinates into 4×64 -bit limbs (25 public inputs), and dispatches to the verifier.
- `src/NoteBindingGroth16Verifier.sol` — placeholder stub (fail-closed: returns `false`) providing the expected `verifyProof(uint[2],uint[2][2],uint[2],uint[25])` interface. Will be replaced by `snarkjs zkkey export solidityverifier` when the trusted setup runs.

Both contracts compile cleanly with `forge build`.

2.5 Tests

- `test/NoteBindingVerifier.t.sol` — 4 tests exercising proof-length gating and limb decomposition (all pass).
- Existing `test/NotesCoupledA1.t.sol` (8 tests) and `test/NotesCoupledA2.t.sol` (10 tests) continue to pass with the stub binding verifier.

2.6 A1-Layout Circuit (`circuits/note_binding_a1.circom`)

An A1 note’s `idHash` commits $(eNote, m_issuer, \sigma)$ — the public-issuer payload, no second ciphertext — so the A2 circuit’s relation is not constructible for A1 notes. The A1 sibling ties the note’s *own* value ciphertext instead: every point in the relation is a known multiple of G ($eNote.C = (v + rn \cdot m_rec) \cdot G$, $eEnc.C = (m_rec + t \cdot m_rec) \cdot G$), so the circuit needs no witnessed point limbs at all. The note face v is a *public* input (the contract passes the spend’s *face*): ElGamal is not key-committing, and pinning the plaintext publicly is what makes the addressed identity unique (Proofs, Theorem 12, A1 statement).

Result: ~ 2.86 M non-linear constraints, 558 MB R1CS, 26 public inputs (face at index 1). Same toolchain requirements as the A2 circuit (C++ witness calculator, `-fno-strict-aliasing`, 64 MB stack, `-O2`, `pot22`); same rapidsnark proving path.

Circuit constraints:

Constraint	Description	EC ops
(1)	nullifier = Poseidon3(rho, idHash, 4242)	—
(2)	idHash = Poseidon8(eNote, mIss, sigR, sigS)	—
(3)	eNote.R = rn*G	rn*G (fixed-base)
(4)	eNote.C = u*G, u = v + rn·m _{rec}	u*G (fixed-base)
(5)	eEnc.R = t*G	t*G (fixed-base)
(6)	eEnc.C = m _{rec} *G + tm*G, tm = t·m _{rec}	2 fixed-base + 1 add
(7)	P _I = m _{rec} *G + b*H	b*H (fixed-base) + 1 add

2.7 Supporting Scripts

- `scripts/snark/gen_note_binding_input.py` — generates test witness JSON from random valid values.
- `scripts/snark/gen_note_binding_a1_input.py` — A1 sibling; delegates to the production witness builder.
- `scripts/snark/gen_bn254_g_pows.py` — generates the G-powers table.
- `scripts/snark/setup_note_binding.sh` — full setup pipeline (circuit compilation, witness generation, Groth16 setup, verifier export, test vector generation).
- `scripts/snark/setup_note_binding_a1.sh` — A1 sibling (make `nix-snark-note-binding-a1`).
- `alberta_buck/wallet/note_binding.py` — Python prover module (`make_note_binding_witness`, `prove_note_binding`, `make_note_binding_a1_witness`, `prove_note_binding_a1`).

2.8 Witness Generation — ROOT CAUSE FOUND AND FIXED

The "Assertion failed: (Fr_{isTrue}(&expaux[0]))" abort on the trivially-true `dummy * dummy == 0` constraint was NOT a circuit bug, a circom-lib BN254 incompatibility, or a powers-table error. It was **strict-aliasing undefined behaviour** in circom's generic (`--no_asm`) `fr.cpp` field-arithmetic runtime, miscompiled by gcc at `-O3`.

2.8.1 Diagnosis trail

1. The G-powers table and `ScalarMulG()` were validated independently: the small `test_sg.circom` harness compiled to WASM produces `s*G` matching the Python wallet's EC implementation exactly, for both `s = 1` and a random 254-bit scalar. The circuit logic is correct; only the C++ witness path failed.
2. Instrumenting the failing assert showed `Fr_eq(0, 0)` returning **false** for the operand-type combination (LONG-normal 0, SHORT-MONTGOMERY 0) — i.e. comparing the result of `Fr_mul(dummy, dummy)` against the constant 0.
3. The same `fr.o` object linked into a minimal standalone test returned the **correct** result for byte-identical inputs — and adding a `printf` inside the failing helper "fixed" it: a classic optimizer/UB heisenbug.
4. Mechanism: the generic `fr.cpp` passes `uint64_t*` limb arrays into GMP's **inline** `mpn_cmp` / `mpn_add_n` / `mpn_sub_n`, whose parameter type is `mp_limb_t*` (= `unsigned long*` on macOS). `unsigned long` and `unsigned long long` are distinct types for C++ aliasing purposes

even though both are 64-bit, and `-fpermissive` lets the conversion compile. `gcc -O3` then assumes the `uint64_t` stores (e.g. filling the `Fr_toLongNormal` temporary inside `req_l1ns2 / req_s112n`) cannot alias the `unsigned long` loads in the inlined comparison, and reads **stale stack garbage** instead. Whether the garbage happens to be zero depends on what the caller left on the stack — which is why small tests passed and the real circuit failed, and why the failure appeared at the earliest constraint involving a mixed-representation compare.

2.8.2 Fix

Two build requirements, both now encoded in `scripts/snark/setup_note_binding.sh`:

1. Compile the generated C++ with `-fno-strict-aliasing` (in addition to `-fpermissive`). This makes gcc honour the cross-type limb accesses and restores correct field arithmetic.
2. Run the witness generator with `ulimit -s 65520` (64 MB stack). The generated template-run functions allocate the expanded G-powers table in ~5.4 MB stack frames (`FrElement lvar[67614]` plus `lvarcall[67598]` per `ScalarMulG`); nested template runs exceed the macOS 8 MB default main-thread stack.

With both applied, the `test_sg` C++ witness generator produces a witness whose `ScalarMulG` output matches the Python EC reference (`CPP-WITNESS-MATCH`), on the same input that previously aborted. The full `note_binding` circuit then generates its 196 MB witness in ~9 seconds (5.9M wires; the C++ generator itself compiles in ~9 minutes at `-O3`), and `snarkjs wtns check` confirms the witness satisfies all ~5.9M R1CS constraints (`WITNESS IS CORRECT`, 27 s).

The WASM path remains unusable at full-circuit size ("memory access out of bounds", ~5.9M wires) and is no longer needed: the pipeline is C++-witness + `rapidsnark-prove`.

2.9 rapidsnark Integration

`rapidsnark v0.0.8` prebuilt macOS-arm64 binaries are vendored as `lib/rapidsnark-macOS-arm64-v0.0.8.zip` (Makefile rule downloads it; `make rapidsnark` unzips to `lib/rapidsnark-macOS-arm64-v0.0.8/bin/`). Contents: `prover`, `verifier`, `test_prover` plus static libs and headers.

- `prover <zkey> <wtns> <proof.json> <public.json>` replaces `snarkjs groth16 prove` (seconds instead of many minutes at 2.4M constraints).
- `verifier` cross-checks the proof off-chain alongside `snarkjs groth16 verify`.
- `rapidsnark` does **not** do witness generation or trusted setup; the circom C++ calculator (fixed above) and `snarkjs` keep those roles.
- No `flake.nix` change needed — the prebuilt binaries are used as-is.

2.10 Groth16 Trusted Setup — COMPLETE; two more toolchain discoveries

The full pipeline now runs end to end (`make nix-snark-note-binding`) and the real verifier is exported and on-chain verified. Getting there surfaced two more toolchain findings, both now encoded in the scripts:

2.10.1 Discovery 1 — snarkjs sizes the Groth16 domain on TOTAL constraints

The circuit's ~2.4M *non-linear* constraints exceeded pot20, so a **pot22** dev ptau (4.19M domain) was generated (**snarkjs ptn/ptc/pt2** with dev entropy; ~4.8 GB final file, several hours of **pt2**). The first setup then failed anyway: at circom's default --01 the R1CS carries the 2.4M non-linear constraints PLUS ~3.5M *linear* ones, and **snarkjs groth16 setup** sizes the domain on the **total** (6,137,856 -> needs pot23).

Fix: compile with --02 (full linear-constraint simplification). Result: **2,407,838 constraints, 0 linear**, R1CS 748 MB -> 471 MB, witness 196 MB -> 73 MB (2.4M wires), and the --02 pass costs only ~90 s of compile time. pot22 fits with 1.8M constraints of headroom. (Caught by a new fail-fast guard in the setup script – **snarkjs** exits 0 even on "circuit too big".)

2.10.2 Discovery 2 — snarkjs 0.7.5 exports the VK already EIP-197-ordered

With the verifier exported and the proof verifying off-chain (snarkjs AND rapidsnark), the on-chain test still rejected it – **ecpairing** returned **PrecompileError** (malformed G2 point). Root cause: **fix_verifier_g2.py** was written for an older snarkjs whose Solidity template embedded the VK G2 constants (beta/gamma/delta) in [x_re, x_im, y_re, y_im] order; **snarkjs 0.7.5 already emits them EIP-197-ordered**, so swapping them corrupts the verifier. Only the *proof-B* swap is still wanted (the repo stores/packs **pi_b** in proof.json natural order – the caller-side swap that **snarkjs generatecall** would otherwise do).

Fix: **fix_verifier_g2.py --b-only** (new flag; default behaviour unchanged for the older committed verifiers), used by **setup_note_binding.sh**. **Latent caveat for the other circuits:** re-running e.g. the gltie setup under snarkjs 0.7.5 with the default full fix would corrupt *its* verifier the same way – make **snark-test-regression** exists to catch exactly this.

2.10.3 Final results

Step	Result
snarkjs wtns check	WITNESS IS CORRECT (2.4M constraints)
groth16 setup (pot22)	1.3 GB zkey
Prove (rapidsnark)	seconds (snarkjs: ~33 s)
Verify off-chain	snarkjs OK + rapidsnark "Valid proof"
Verify on-chain (raw)	~501K gas
Verify on-chain (adapter)	~508K gas (25 public inputs derived on chain)
NoteBindingVerifier.t.sol	9/9 pass (real proof + tamper rejections)
make nix-test	393 pass; 8 pre-existing env failures (uniswap artifacts / manual fixture), none r
make nix-test-python	329 pass; 2 unrelated failures (uniswap-artifact env test; one known-flaky oracle

src/NoteBindingGroth16Verifier.sol is now the real generated verifier (proof-B swap applied); the fail-closed stub is gone. The wallet's **prove_note_binding** was reprinted from the unusable WASM path to the C++ witness generator (64 MB stack) + rapidsnark with snarkjs fallback.

3 Remaining Issues

3.1 flake.nix Dependencies

Present in **flake.nix** and sufficient:

- `gcc14` — C++ compiler (with `-fno-strict-aliasing`; see above)
- `nlohmann_json` — JSON parsing
- `gmp` — big-integer arithmetic
- `nasm` — x86-64 assembler (not needed with `--no_asm`; retained)
- `pkg-config` — build-config discovery

rapidsnark is intentionally **not** in `flake.nix`; the prebuilt release binaries under `lib/` are used directly (see rapidsnark Integration).

4 File Inventory

File	Status
<code>circuits/note_binding.circom</code>	Done
<code>circuits/note_binding_a1.circom</code>	Done
<code>circuits/ec/bn254_g_scalarmul.circom</code>	Done
<code>circuits/ec/powers/bn254_g_pows.circom</code>	Done (generated)
<code>scripts/snark/gen_note_binding_input.py</code>	Done
<code>scripts/snark/gen_note_binding_a1_input.py</code>	Done
<code>scripts/snark/gen_bn254_g_pows.py</code>	Done
<code>scripts/snark/setup_note_binding.sh</code>	Done (rewritten)
<code>scripts/snark/setup_note_binding_a1.sh</code>	Done
<code>scripts/snark/debug_witness.js</code>	Debug
<code>lib/rapidsnark-macOS-arm64-v0.0.8/</code>	Vendored binaries
<code>src/INoteBindingVerifier.sol</code>	Done (A1 + A2)
<code>src/StubNoteBindingVerifier.sol</code>	Done (A1 + A2)
<code>src/NoteBindingVerifierAdapter.sol</code>	Done (A1 + A2)
<code>src/NoteBindingGroth16Verifier.sol</code>	Done (generated)
<code>src/NoteBindingA1Groth16Verifier.sol</code>	Done (generated)
<code>test/NoteBindingVerifier.t.sol</code>	Done (A1 + A2)
<code>test/NotesCoupledA1.t.sol</code>	Done (prior)
<code>test/NotesCoupledA2.t.sol</code>	Done (prior)
<code>alberta_buck/wallet/note_binding.py</code>	Done
<code>alberta_buck/wallet/unilateral_a2.py</code>	Modified
<code>alberta_buck/wallet/unilateral_a1.py</code>	Modified
<code>Makefile</code>	Modified

5 Build Commands

```
# Regenerate BN254 G-powers table (needed after wallet EC changes)
make nix-snark-g-pows
```

```
# Fetch + unzip the rapidsnark prover/verifier binaries into lib/
make rapidsnark
```

```
# Compile circuit, generate + check witness, run full Groth16 setup,
```

```
# export src/NoteBindingGroth16Verifier.sol and Forge test vectors
make nix-snark-note-binding
```

```
# Run all Solidity tests
make nix-test
```

```
# Run Python wallet tests
make nix-test-python
```